

SKILL WEEK 5

QUESTION 1

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

Aim

To understand and implement **single-quote handling in Bash** for preserving literal content, preventing variable expansion, storing quoted strings, validating parsing results, and testing edge cases.

1. Single Quote vs Double Quote

Terminal

```
amrutha@blue:~/OSSP/week5_s$ pwd
/home/amrutha/OSSP/week5_s
amrutha@blue:~/OSSP/week5_s$ name="amru"
amrutha@blue:~/OSSP/week5_s$ echo 'Hello $name'
Hello $name
amrutha@blue:~/OSSP/week5_s$ echo "Hello $name"
Hello amru
```

Output

Hello \$name

Hello amru

Observation:

Single quotes preserve \$name literally, while double quotes allow variable expansion.

2. Single Quotes Preserve Literal Content

Terminal

```
amrutha@blue:~/OSSP/week5_s$ echo '$HOME'
$HOME
amrutha@blue:~/OSSP/week5_s$ echo '$USER'
$USER
```

Output

\$HOME

\$USER

Observation:

The contents inside single quotes are treated literally. Variables and command substitutions are not expanded.

3. Store Quoted Strings in Variables

Terminal

```
amrutha@blue:~/OSSP/week5_s$ message='OSSP Week 5 Skill Assignment'
amrutha@blue:~/OSSP/week5_s$ echo "$message"
OSSP Week 5 Skill Assignment
amrutha@blue:~/OSSP/week5_s$ message='Linux shell script'
amrutha@blue:~/OSSP/week5_s$ echo "$message"
Linux shell script
amrutha@blue:~/OSSP/week5_s$ echo '$message'
$message
```

Output

OSSP Week 5 Skill Assignment

Linux shell script

\$message

4. Parsing Validation

Terminal

```
amrutha@blue:~/OSSP/week5_s$ nano valid.sh
amrutha@blue:~/OSSP/week5_s$ bash -n valid.sh
amrutha@blue:~/OSSP/week5_s$ bash valid.sh
Hello amru
amrutha@blue:~/OSSP/week5_s$ nano invalid.sh
amrutha@blue:~/OSSP/week5_s$ bash -n invalid.sh
invalid.sh: line 1: unexpected EOF while looking for matching `"'
amrutha@blue:~/OSSP/week5_s$
```

Output

For valid quoted command:

Hello amru

For an invalid quoted command:

invalid.sh: line 1: unexpected EOF while looking for matching `"'

Observation:

bash -n checks the syntax without executing the script and identifies unmatched quotation marks.

Result

Single-quote handling was successfully implemented. Literal content was preserved, variable expansion was prevented, quoted strings were stored and tested, and parsing errors caused by incorrect quotation were identified.

QUESTION 2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Aim

To understand and implement **double-quote handling in Bash** for preserving spaces, allowing variable expansion, parsing nested tokens, validating outputs, and executing quoted commands.

Program / Commands

1. Double Quotes and Variable Expansion

Terminal

```
amrutha@blue:~/OSSP/week5_s$ name="amrutha"  
amrutha@blue:~/OSSP/week5_s$ echo "Hello $name"  
Hello amrutha
```

Output

Hello amrutha

Observation:

Unlike single quotes, double quotes allow variables such as \$name to be expanded.

2. Preserve Spaces with Double Quotes

Terminal

```
amrutha@blue:~/OSSP/week5_s$ text="Linux Shell Programming"  
amrutha@blue:~/OSSP/week5_s$ printf '<%s>\n' "$text"  
<Linux Shell Programming>  
amrutha@blue:~/OSSP/week5_s$ printf '<%s>\n' $text  
<Linux>  
<Shell>  
<Programming>
```

Output

<Linux Shell Programming>

<Linux>

<Shell>

<Programming>

Observation:

Using "\$text" preserves the complete string as one argument, including its spaces.

3. Argument Parsing

Create:

```
amrutha@blue:~/OSSP/week5_s$ show_args() {  
    echo "Number of arguments: $#"  
    for arg in "$@"; do  
        echo "Argument: [$arg]"  
    done  
}
```

Now:

```
amrutha@blue:~/OSSP/week5_s$ show_args "$text"  
Number of arguments: 1  
Argument: [Linux Shell Programming]  
amrutha@blue:~/OSSP/week5_s$ show_args $text  
Number of arguments: 3  
Argument: [Linux]  
Argument: [Shell]  
Argument: [Programming]
```

Output

With double quotes:

Number of arguments: 1

Argument: [Linux Shell Programming]

Without double quotes:

Without double quotes:

Number of arguments: 3

Argument: [Linux]

Argument: [Shell]

Argument: [Programming]

Observation:

Double quotes preserve the spaces and pass the complete value as a single argument.

4. Nested Tokens / Command Substitution

Terminal

```
amrutha@blue:~/OSSP/week5_s$ echo "Current user: $USER"
                                echo "Current directory: $(pwd)"
                                echo "Current date: $(date)"
```

Output

```
Current user: amrutha
Current directory: /home/amrutha/OSSP/week5_s
Current date: Sat Sep 19 03:37:33 UTC 2026
```

Observation:

Variables are expanded inside double quotes, and command substitution using `$(...)` is also evaluated.

5. Quoted Command

Terminal

```
amrutha@blue:~/OSSP/week5_s$ command='echo "Hello from a quoted command"'
amrutha@blue:~/OSSP/week5_s$ echo "$command"
echo "Hello from a quoted command"
amrutha@blue:~/OSSP/week5_s$ bash -c "$command"
Hello from a quoted command
```

Output

echo "Hello from a quoted command"

Hello from a quoted command

Observation:

The command can be stored as a quoted string and then executed using `bash -c`.

6. Validate Output / Parsing

Check the syntax of the script:

```
amrutha@blue:~/OSSP/week5_s$ bash -n week5_quotes.sh
```

Then execute:

```
amrutha@blue:~/OSSP/week5_s$ bash week5_quotes.sh
Hello $name
Hello amru
$HOME
$USER
<Linux Shell Programming>
<Linux>
<Shell>
<Programming>
Current user: amrutha
Current directory: /home/amrutha/OSSP/week5_s
Current date: Sat Sep 19 04:01:40 UTC 2026
echo "Hello from a quoted command"
Hello from a quoted command
```

Result

Double-quote handling was successfully implemented. Spaces were preserved, variables were expanded, nested tokens and command substitution were evaluated, quoted commands were executed, and the Bash script was validated using `bash -n`.